OpenLocalAgent: Transfer and Task Completion in Small Tool-Calling Models

Anonymous Author(s)
Affiliation
Address
email

Abstract

Tool-calling research centres on large language models, leaving models in the tens of millions of parameters underexamined. We give an end-to-end recipe for a 10 to 95M parameter caller, spanning pretraining through episode supervision, and price its ingredients against nineteen released checkpoints under one harness on ten public evaluation suites, including explicitly labelled text-only, filtered and first-action projections. Data and additional pretraining compute transfer selectively: an imported split improves predominantly the suites whose tool inventory it shares, while longer pretraining improves selected suites with no imported rows. Under the maps tested, coordinate-preserving projections from larger models do not improve suite scores over random initialisation, though same-shape weights transfer strongly. Single-call accuracy does not yield task completion in our simulator: all single-call-trained checkpoints complete zero multi-step episodes. Episode supervision at a small mixture share lifts completion from zero to double-digit rates with no consistent single-call cost across the two scope groups, and an oracle decomposition identifies argument fidelity, rather than tool ordering, as the larger remaining bottleneck in this simulator.

1 Introduction

Research on tool-calling agents advances on the back of large language models, and open releases follow it: general families commonly bottom out near 0.6B [9,10], the smallest general instruction baseline here is 135M [8], and the 45M needle2 [37] is a rare specialist. Yet many device actions are short, typed calls rather than open-ended generation, making the 10M–95M regime plausible and largely unmeasured. We study it stage by stage rather than report one final checkpoint.

The practical constraint creates a second problem. A custom edge-sized shape forfeits compatible released weights, while a catalog-conditioned caller must choose from tool descriptions supplied at runtime rather than a fixed inventory compiled into its output layer. Pretraining must therefore support catalog reading, fine-tuning must teach call syntax and arguments, and episode supervision must turn isolated calls into state-conditioned behaviour. We ask what each stage contributes and which knowledge can be imported as data or weights.

Our 10.5M/43M/95.3M family is the instrument. One contract, parser and chance-floor harness scores nineteen released checkpoints on ten public evaluation suites, with projections labelled; controlled comparisons use matched data and step budgets, while shipped recipes and needle2 are separated as unmatched deployment references. An intervention ledger changes one recorded stage at a time, and four-thread server-CPU decode prices the architecture independently of its recipe.

Three findings organise the paper. First, imported data transfers mainly to covered inventories, whereas additional pretraining improves selected suites with no imported rows. Second, the tested

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

coordinate-preserving cross-shape projections fail although same-shape adoption transfers strongly (§4). Third, evaluated single-call checkpoints complete no simulator episodes until sparse episode supervision produces LocalAgent single-checkpoint estimates of 17.6–34.3%; separate one-seed SmolLM2-135M and Qwen3-0.6B controls reach 8.6% and 54.3%. Every model still fails the held-out composition (§5, App. E).

2 Related work

Small agents. Toolformer [52], ReAct [53] and Gorilla [54] establish tool use; Belcak et al. [14] argue that repetitive schema-bound calls suit smaller models, but study 1B–7B. AgentCPM-GUI [4] specialises a larger model for visual device control. Shen et al. [15] and Żywot et al. [16] recover weak callers by adding planner/caller specialists or multi-agent orchestration at inference. Those systems trade orchestration for capability; we hold inference to one policy decision and ask what remains two orders of magnitude lower, across pretraining, SFT and episode supervision.

Edge callers. TinyAgent [13] retrieves tools for 1.1B/7B models; MobileLLM [26] designs sub-billion architectures; Octopus v2 [27] binds a fixed pool to tokens; Hammer [28] masks irrelevant functions. Our models and needle2 [37] instead accept runtime descriptions, but needle2 retrieves five and constrains decoding. Thus fixed-token systems buy latency by compiling the inventory into weights, retrieval systems shorten the prompt, and our experiment isolates free generation from a prompt-supplied, changing catalog. This distinction matters for MCP-style use: adding a tool changes context for our contract, whereas fixed-token callers require vocabulary or weight updates. Our throughput result concerns the former contract on one CPU, not a universal mobile-runtime comparison.

Multi-step use. BUTTON [55] and PARL-MT [56] train multi-turn composition and progress; tau-bench [57] and ToolSandbox [39] evaluate stateful completion. We test whether sparse episode supervision creates completion at 10M–95M, whether single-call accuracy transfers without it, and what the added supervision costs on the original call suites. Unlike those broader interactive environments, our simulator fixes nine tools and executable preconditions so that sequencing, argument fidelity and state change can be separated; it is a controlled diagnostic, not a substitute for their real-world scope.

Weight reuse. Net2Net [18], bert2BERT [19], LiGO [20], HyperCloning [21] and Weight Subcloning [22] transfer across sizes; Wiring Beats Blending [58] shows map structure matters. The former methods preserve functions or learn mappings; tokenizer-aware maps such as WECHSEL [25] address a related embedding mismatch. Our deliberately cheaper test asks whether coordinate-preserving cross-family slices can replace pretraining, then uses same-shape adoption as a positive control. We also test whether fine-tuning update magnitude predicts reusable regions; Yu et al. [46] instead localise individual *super weights*. Our negative result is therefore about the tested slices, not learned alignment, distillation or cross-shape transfer in general (§4, App. B).

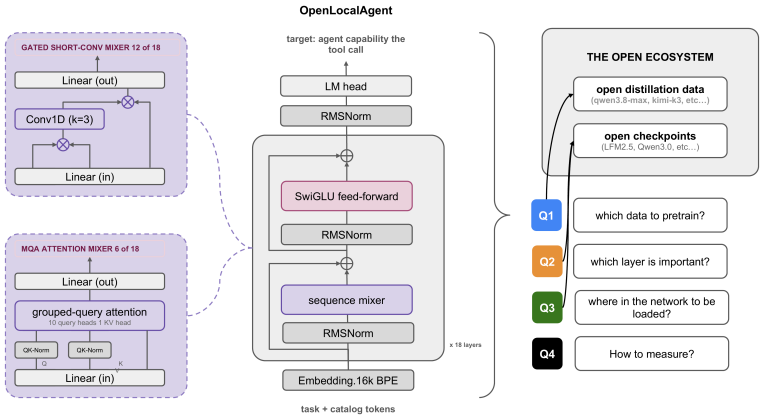

Figure 1: Architecture and study map: a 16k-vocabulary decoder with 12 short-convolution and six MQA blocks. Arrows locate the architecture and data/weight evaluation routes; details: Apps. A and B.

3 One harness, ten public evaluation suites

We first fix the shared measurement contract and compare models; §4 prices data and weight imports, §5 adds task completion, and Apps. A and E give the full projection and episode protocols.

The evaluation pipeline (Q4). Three dataset-author call splits and seven labelled projections/filtered views flow through one prompt contract, parser and scorer (App. A). Catalogs contain 2.1–33 candidates, so we report majority and chance-in-catalog floors [29] and gate differences by measured replicate spread. Panel A uses the same 16,300-row union, sampled rows, 600-step budget and update type; rates and contexts remain family-specific (App. B). Function-name match scores the action, exact match adds arguments; throughput is 64-token decode after a 512-token prefill on four CPU threads.

Models under test. LocalAgent-11M/43M/96M (10.5M, 42.8M, 95.3M parameters) are one decoder family with a 16k vocabulary. Table 1A gives their matched-data/step-budget SFT rows; Panel B gives the stronger five-source shipped SFT after the same 16,000-step pretraining and 700-step midtrain. Every accuracy cell is measured here; needle2's throughput is the exception.

Table 1: Two-panel comparison under one harness: *function-name (exact) match*, %. Panel A compares released, generic and LocalAgent models at the shared 600-step stage; data and step budget match, while family rate/context differences remain (App. B). Only Panel A receives bold/underline maxima. Panel B reports the stronger shipped LocalAgent recipe and vendor-trained needle2 as unmatched deployment references; their accuracy is excluded from ranking. Columns separate dataset-author call splits from labelled projections/filtered views; these are not ten official scores. Throughput is architecture-level. Labels expose text-only, 204/17-row, first-action and name-only scope (App. A); MCP/TS omit exact match. Bold/underline mark per-column name/exact maxima. † needle2's self-reported 2-bit speed; ‡ unavailable cache.

Model	Params (M)	tok/s	dataset-author call splits			study projections / filtered views						
			ToolACE	xLAM	MobAct	Android-text	M2W-text	AgentNet-text	BFCL-204	TB-first	TS-17	MCP-name
Panel A — shared 600-step comparison: released instruction models												
SmolLM2-135M-Inst [8]	135	62	70.8 (24.4)	84.0 (37.7)	66.5 (20.3)	67.0 (42.1)	79.8 (0.8)	53.8 (0.2)	91.2 (32.4)	44.5 (21.3)	5.9	**13.3**
LFM2.5-230M [10]	230	59	89.6 (49.8)	97.0 (61.0)	94.9 (47.0)	80.5 (48.9)	80.2 (1.2)	70.1 (0.1)	92.2 (46.6)	55.5 (35.4)	11.8	3.8
Granite-4.0-H-350M [36]	340	30	89.0 (40.7)	94.0 (56.0)	88.4 (36.9)	67.0 (45.4)	80.2 (1.2)	58.3 (0.1)	89.7 (44.6)	56.9 (36.5)	23.5	12.3
Granite-4.0-350M [36]	352	—‡	90.5 (48.6)	96.3 (34.3)	96.5 (40.5)	80.2 (48.5)	**81.3** (0.8)	51.5 (0.5)	94.1 (51.5)	62.7 (41.9)	11.8	11.1
LFM2-350M [10]	354	39	89.5 (47.8)	96.0 (55.1)	95.2 (56.2)	80.6 (48.3)	80.2 (0.8)	67.5 (0.1)	93.1 (54.9)	58.1 (37.7)	5.9	5.7
SmolLM2-360M-Inst [8]	362	33	85.2 (40.1)	94.3 (58.9)	97.5 (39.3)	78.9 (47.3)	80.2 (0.8)	22.6 (1.0)	95.6 (55.9)	52.3 (32.2)	11.8	8.5
Qwen2.5-0.5B-Inst [9]	494	30	88.7 (47.7)	96.2 (27.8)	98.6 (52.9)	80.5 (48.2)	80.2 (0.4)	26.9 (0.7)	94.1 (52.0)	59.1 (40.2)	23.5	13.1
Qwen2.5-Coder-0.5B [9]	494	28	91.8 (52.3)	96.6 (60.8)	98.8 (53.9)	80.8 (49.3)	80.2 (0.0)	54.8 (0.4)	95.6 (56.9)	59.4 (40.9)	29.4	11.5
danube3-500M-chat [11]	514	30	86.7 (34.6)	93.5 (36.0)	94.5 (20.4)	80.3 (49.0)	78.6 (1.6)	69.1 (0.3)	91.7 (42.2)	58.8 (37.1)	5.9	8.7
Qwen3-0.6B [9]	596	21	**92.2** (54.5)	**97.4** (58.1)	**99.6** (64.1)	**80.9** (50.3)	80.2 (0.8)	52.1 (1.3)	**97.1** (61.3)	65.4 (44.7)	**47.1**	9.9
LFM2-700M [10]	742	21	91.3 (51.8)	96.7 (57.5)	94.8 (62.5)	67.5 (49.8)	80.2 (1.6)	64.9 (0.3)	94.1 (58.3)	**66.7** (48.0)	5.9	12.5
Generic pretrained backbones under the same stage (§4)												
Pythia-70M [47]	70	208	17.3 (0.9)	20.8 (0.7)	1.5 (0.0)	37.1 (8.8)	29.8 (0.0)	21.6 (0.0)	23.5 (0.0)	5.4 (1.3)	5.9	0.0
DistilGPT-2 [49]	82	118	52.2 (7.3)	59.5 (8.7)	48.5 (3.2)	49.0 (18.9)	80.2 (0.4)	**72.1** (0.0)	74.5 (6.4)	31.3 (9.0)	23.5	3.8
GPT-2 [49]	124	83	58.9 (13.2)	69.9 (15.1)	49.9 (17.0)	50.8 (27.0)	60.3 (0.0)	63.2 (0.1)	83.8 (12.3)	30.0 (7.9)	5.9	6.9
Pythia-160M [47]	162	78	48.1 (10.1)	55.4 (6.2)	11.1 (3.2)	67.3 (18.9)	44.4 (0.0)	58.2 (0.1)	62.3 (6.4)	15.4 (6.7)	5.9	1.0
Mamba-370M [48]	372	25	67.1 (26.9)	79.5 (36.8)	41.5 (20.4)	54.2 (30.9)	80.2 (0.4)	65.9 (0.1)	80.9 (34.8)	39.9 (21.8)	5.9	7.1
Pythia-410M [47]	405	31	82.2 (34.0)	90.6 (46.9)	93.2 (14.8)	80.3 (45.7)	81.0 (1.2)	60.2 (0.1)	90.7 (41.2)	50.2 (32.9)	11.8	6.5
Mamba-790M [48]	793	16	72.8 (31.5)	86.7 (46.9)	89.5 (39.3)	60.7 (30.1)	80.2 (0.0)	56.9 (0.1)	89.7 (43.6)	41.3 (26.8)	17.6	8.5
LocalAgent under matched data and step budget												
LocalAgent-11M (matched)	11	604	35.7 (5.5)	43.8 (7.9)	22.2 (1.7)	52.1 (26.2)	80.2 (1.6)	59.1 (0.0)	44.6 (2.9)	12.7 (1.6)	0.0	0.8
LocalAgent-43M (matched)	43	173	41.2 (10.2)	52.2 (12.8)	29.8 (0.4)	74.2 (36.8)	80.2 (1.6)	37.8 (0.0)	52.9 (7.4)	24.5 (4.2)	0.0	3.6
LocalAgent-96M (matched)	95	85	43.6 (15.4)	56.7 (17.6)	17.5 (0.5)	79.2 (44.5)	80.2 (1.2)	68.7 (0.0)	64.2 (12.7)	23.6 (3.9)	0.0	3.0
Panel B — unmatched deployment recipes (reference only; excluded from maxima)												
LocalAgent-11M (shipped)	11	604	33.4 (9.6)	73.1 (34.1)	80.6 (67.2)	67.1 (39.4)	80.2 (0.8)	55.5 (0.0)	49.0 (8.8)	28.2 (13.1)	0.0	4.2
LocalAgent-43M (shipped)	43	173	52.5 (20.8)	91.3 (49.3)	81.2 (69.5)	65.7 (40.8)	80.2 (0.8)	38.0 (0.4)	69.1 (14.7)	41.0 (20.8)	0.0	2.0
LocalAgent-96M (shipped)	95	85	58.5 (25.0)	90.1 (50.2)	81.9 (70.2)	67.4 (41.9)	80.2 (0.8)	47.3 (0.0)	66.2 (15.2)	41.2 (20.7)	0.0	2.6
needle2 [37] (vendor)	45	469†	61.2 (34.2)	89.4 (40.0)	71.3 (28.4)	23.3 (15.3)	11.5 (0.0)	8.9 (0.0)	73.5 (23.5)	24.7 (14.3)	70.6	2.4

What the controlled panel establishes. Within the eleven-model released-instruction block, fine-tuning improves 103 of 110 model-suite cells; the separate generic-backbone block is not included in that count. Within Panel A, Qwen3-0.6B has the best function-name match on six of the ten suites; Granite-4.0-350M takes Mind2Web; DistilGPT-2 takes AgentNet; LFM2-700M takes ToolBench; SmolLM2-135M-Inst takes MCP-A. ToolBench makes the gap clear: chance is 22.9%, every instruction-tuned release clears it (44.5–66.7%), while matched LocalAgent-96M reaches 23.6%. Mind2Web remains majority-floor dominated (78.6–81.3; ours 80.2). We report no pooled rank: matched LocalAgent-96M scores 39.3/45.6 on the author/projection scope means, near Pythia-160M's 38.2/36.4 and below DistilGPT-2's 53.4/47.8. Panel B's shipped 96M reaches 76.8/43.5, but that gain is recipe evidence, not an architecture comparison; needle2 is likewise external. The robust architecture result is throughput: 604 tok/s for 11M versus about 30 for 0.5B models on four server-CPU threads; needle2's self-reported 2-bit speed is incomparable.

4 Data, weights, or neither: the intervention ledger

With the baseline tradeoffs established, each intervention in Table 2 runs against the control isolating it, chain and hyper-parameters fixed. Terms: *adoption* copies weights between identical shapes, *projection* crosses a shape change; *union* arms train on the union corpus, *wide* adds xLAM's split, *+teacher* relabels it.

Generic backbones as a control for purpose pretraining. Under the matched stage, Pythia's author/projection means scale 13.2/17.6 (70M), 38.2/36.4 (160M), 88.7/54.4 (410M); LocalAgent-96M reaches 39.3/45.6 and DistilGPT-2-82M 53.4/47.8. Only ToolBench separates LocalAgent from Pythia-160M beyond replicate spread (+8.2), with LocalAgent still near chance. Thus the matched 96M sits near Pythia-160M and below Pythia-410M, not at a universal purpose-pretraining advantage; corpus, tokenizer, architecture and budget still co-vary.

Covered-inventory data transfers; tested cross-shape slices do not. Importing xLAM's 18,000-row split moves xLAM +26.8, 3.4× its spread, from below to above chance. Teacher relabelling recovers device-control points only within spread. By contrast, no tested coordinate-preserving projection clears an open-catalog spread; learned mappings such as LiGO [20] remain untested.

Inventory coverage dominates the imported-data effect. The split covers 99.4% of xLAM evaluation names but 1.0% of ToolACE and 0% of ToolBench gold names; only xLAM moves. Renaming costs the 96M rung under four points but the 11M rung thirty-two (App. B), making name lookup a size-dependent shortcut [17,28]. Longer pretraining moves ToolACE +12.1 and ToolBench +9.0 beyond replicate spread; BFCL's +11.7 remains within its wide spread and only 1.8 points above its chance floor, so chance clearance is not established there. A fresh-corpus control reproduces ToolACE but gives back 8.1 points on ToolBench, identifying the latter gain as repetition-keyed (App. C). ToolACE's sequential 39.2→51.3→59.3 path does not establish additivity: coverage-associated gain is +4.0 at 1,599 steps and +8.0 at 16,000, but the arms are not a matched 2×2 factorial. Thus ToolACE is the corpus-robust evidence that pretraining can improve a suite with no imported rows; BFCL and ToolBench do not independently establish that mechanism (App. A).

Table 2: Function-name-match change against each row's stated control. Bold marks an absolute change that exceeds the measured replicate spread (five runs, four seeds); Mind2Web is floor-dominated and never bolded. Open models appear only as donors or teachers. Full noise and multiplicity analysis: App. A.

Model	Intervention (vs our model's own control)	AndroidCtrl	Mind2Web	AgentNet	ToolACE	xLAM	BFCL	ToolBench	ToolSB	MCP-A	MobAct
Import data into our model's SFT											
11M	add the xLAM train split to the union corpus	-12.3	-0.4	-16.2	-3.4	**+26.8**	-1.0	-0.9	+0.0	**+2.8**	+2.7
11M	relabel that corpus with the strongest fine-tuned teacher	+10.2	+0.8	+3.4	+3.5	+5.7	+4.4	-2.6	+0.0	**-4.2**	-6.7
96M	the two imports above together	-2.2	+0.0	+21.3	+4.0	**+44.6**	+6.4	**+9.3**	+0.0	-1.8	**+22.6**
96M	add Toucan real-MCP trajectories [42]	-12.9	+0.0	-13.7	+2.8	+0.9	-1.0	+1.3	+0.0	+1.0	-1.6
96M	add schema-synth conversations from the ToolSandbox inventory	-1.5	-0.8	**-32.4**	-0.3	-0.9	-2.9	-0.9	+0.0	+0.4	-0.2
11M	add the ToolBench train split (70% name overlap)	-2.3	+0.0	**+34.4**	-3.6	-3.3	+1.0	**+22.0**	+0.0	**+3.0**	-1.8
11M	add the Mobile-Actions split its recipe had omitted	-1.0	-0.4	+11.2	**-9.3**	**-8.7**	-8.8	+0.9	+0.0	+0.0	**+64.1**
96M	add device-assistant data (0% exact name overlap)	+12.4	-0.4	+8.8	-4.8	-2.5	-1.0	-3.8	+0.0	-0.4	+0.8
11M	all four splits above at once	-7.4	+0.0	**-25.8**	-3.0	-5.0	-2.0	**+15.1**	+0.0	+1.2	**+65.9**
96M	all four splits above at once	+0.3	+0.0	-15.5	-0.3	+0.2	-1.5	**+9.3**	+0.0	-0.8	-1.1
Import weights into our model											
11M	initialise from a projected Qwen2.5-0.5B (cross-family)	+1.9	+2.8	+0.1	+0.1	-0.1	+0.0	+0.0	+0.0	+0.0	+0.0
11M	adopt an identical-shape sibling's weights (positive control)	**+16.3**	+1.6	**+42.6**	**+32.3**	**+36.7**	**+46.1**	**+9.9**	+0.0	**+2.2**	**+20.0**
11M	train under a donor's per-module learning-rate profile	-3.2	+0.0	-12.9	-0.5	+0.2	+1.5	+1.7	+0.0	+0.8	+4.9
Scale the model											
11M→96M	grow the architecture, recipe unchanged	+0.8	+0.4	-1.8	+5.2	+7.3	+5.4	-2.5	+0.0	+0.6	-4.2
96M	pretraining budget 1,599 → 16,000 steps	+9.0	+1.6	+4.3	**+12.1**	+2.9	+11.8	**+9.0**	+0.0	**+2.2**	+4.6
96M	coverage split and a stronger teacher on the 16,000-step backbone	-8.5	-0.8	-6.2	**+8.0**	+0.0	+6.4	-5.5	+0.0	-1.2	-3.7
11M	pretraining budget 1,599 → 16,000 steps under the five-source SFT	+11.2	+0.0	**+51.1**	+6.2	**+9.9**	+7.8	+4.2	+0.0	**+2.8**	-1.1
Swap the pretraining stream											
96M	curated web → dialogue and distillation text, SFT matched	+0.4	+0.0	+1.5	+4.5	+1.3	+6.4	-5.2	+0.0	-0.8	-0.5
96M	the swapped stream under the five-source SFT	+0.4	+0.8	+21.8	+2.6	+1.5	+2.5	-2.8	+0.0	-0.8	+0.9
11M	the swapped stream under the five-source SFT at 10.5M	-2.9	+0.4	-17.1	+3.8	+0.7	-4.4	-1.2	+0.0	-0.2	-1.4
43M	the swapped stream under the five-source SFT at 43M	+1.5	+0.4	-18.9	-2.5	-4.3	-13.2	**-8.4**	+0.0	-0.2	-0.2
	replicate spread (noise band, 5 runs, 4 seeds)	±14.8	±2.4	±25.3	±6.5	±7.8	±16.7	±6.4	±5.9	±2.2	±19.7

Same-shape weights remain reusable. Against random initialisation, the whole backbone gains 29.7/17.0 on author/projection means; attention alone retains 76%/38%, while feed-forward gains only 0.8/3.0. Yet fine-tuning writes hardest in feed-forward projections, and the least-written layer third recovers 85% of all-layer gain against 65% for the most-written third. Update magnitude therefore does not predict useful adoption or adaptation here. Every cross-shape donor (1.4×–6.3×)

keeps device-control accuracy but loses open-catalog performance; full anatomy and controls are in App. B.

5 From calls to task completion, and what the recipe costs

From a correct call to a completed task. Tables 1 and 2 score one call, but an agent acts against state produced by its earlier calls. We therefore evaluate policies in a stateful simulator with real tool preconditions and twelve error codes: the policy sees the goal, current state and last error, and success is a path-independent check on the final state. The suite contains 210 held-out tasks over seven families and disjoint slot vocabularies; oracle replay completes 100.0% and uniform sampling 0.5%. Single-call accuracy does not transfer: every Table 1 LocalAgent checkpoint completes 0.0%. The 96M checkpoint nevertheless emits a schema-valid call on 88% of steps and changes state on 50%, showing that legality is not goal direction. The shared union's 1,606 synthetic multi-turn conversations serialize calls and replies but omit the simulator's goal/state/last-error decision observation; App. E's episode rows are state-aligned decisions, which is why the former do not supply this supervision.

Table 3: Episode task completion in the stateful simulator (%, 210 tasks). The three 12.5% LocalAgent cells are one checkpoint each, not three-seed means. Schema/Moved: fraction of steps schema-valid / state-changing. Held-out: a never-trained three-goal composition. Full profiles, open-model controls, seeds and paired tests: App. E.

Policy	Episode	Schema	Moved	Held-out
Single-call checkpoint, 96M	0.0	88	50	0.0
12.5% episodes, 11M	20.5	82	59	0.0
12.5% episodes, 43M	34.3	87	81	0.0
12.5% episodes, 96M	17.6	71	56	0.0
38% episodes, 96M	25.2	87	77	0.0

Episode supervision adds completion, but only inside the trained task family. Re-running the same fine-tuning with episode decisions at a 12.5% mixture share reaches single-checkpoint estimates of 20.5%, 34.3% and 17.6% completion at 11M, 43M and 96M. Only the 11M sparse and 96M dense procedures have three-seed repeats; these values establish no size ordering. On the dataset-author call splits, the descriptive mean changes +2.3, -0.5 and +1.4 points; on the study projections / filtered views it changes +1.8, +3.6 and +4.2. Thus there is no consistent single-call cost at the 12.5% share, but one suite-level regression exceeds its replicate spread (full ledger in App. E), so this is not a claim of no interference. The same intervention moves SmolLM2-135M from 0.0% to 8.6% and Qwen3-0.6B from 0.0% to 54.3% under their own recipes. A diagnostic on the 38% 96M arm separates the remaining ceiling: giving the gold tool sequence leaves completion at 25.2%, whereas giving the gold arguments raises it to 78.6%, making argument fidelity the larger measured bottleneck in this simulator. Recovery episodes account for 23–70% of all successful LocalAgent episodes (42% at 43M), while the explicitly supervised abstention family remains at 0.0% for every reported arm. Every trained checkpoint remains at 0.0% on the never-trained three-goal composition. These are controlled simulator results, not evidence of real-world or compositional task completion; protocol, seed variation, paired tests and error profiles are in App. E.

6 Stress tests, general cost, and claim boundaries

Is the xLAM gain keyed to names? Rewriting only names and gold answers separates catalog reading from lookup. LocalAgent-96M is robust: 90.1 unmodified, 83.4 with opaque identifiers and 86.3 with names rotated; LocalAgent-11M falls from 73.1 to 47.9/41.4. Coverage predicts which suites move, but the 96M result survives name erasure and rotation; memorised lookup is the smaller rung's mechanism, not the imported inventory's only use.

General-suite cost under the earlier LoRA reference is small on average, not uniformly. On HellaSwag, Winogrande, ARC-e/c, OpenBookQA and MMLU, the eleven released baselines' mean change spans -1.8 to +0.7 points, inside the ± 2.9-point sampling error; Granite-4.0 nevertheless loses 5.7–7.3 on ARC-e. These receipts use the old LoRA stage, not Table 1's current full-parameter stage; the custom family was not run here, so no preservation claim follows.

Evidence hierarchy and practical reading. Table 2's within-family pairs hold the recorded chain fixed, but most are single checkpoints and cross-family placement is descriptive. Bold cells exceed a five-run range measured at 10.5M; pool uncertainty and multiplicity are in App. A. Episode task tests compare fixed checkpoints, three-seed ranges compare LocalAgent procedures, and the one-seed open arms establish no scale ordering (App. E). Operationally, covered SFT buys represented inventory, additional pretraining robustly lifts ToolACE without imported rows while the ToolBench gain is repetition-keyed, and same-shape but not tested cross-shape weights remain useful. Stateful execution requires episode decisions and remains bounded by trained task schemas.

Table 4: General-suite change after the earlier LoRA reference recipe (percentage points; 300 identical rows before/after per suite), scored by length-normalised continuation likelihood. Mean is unweighted across the six suites; absolute scores and chance floors are in App. C. These receipts point to `runs/lora/*`; they do not measure Table 1's current full-parameter 2e-5 stage.

Model	ARC-e	ARC-c	HellaSwag	OBQA	Wino	MMLU	mean Δ
SmolLM2-135M-Inst [8]	+2.0	-0.3	+1.3	+0.0	+2.7	-1.3	**+0.73**
LFM2.5-230M [10]	-2.0	-0.4	+1.0	+0.4	-0.3	+0.7	**-0.10**
LFM2-350M [10]	+1.7	+0.7	+0.0	+0.0	+0.7	+0.4	**+0.58**
Granite-4.0-H-350M [36]	-5.7	-1.0	-1.3	-2.4	-2.3	+5.7	**-1.17**
Granite-4.0-350M [36]	-7.3	-3.4	+0.3	+0.3	-2.0	+1.3	**-1.80**
SmolLM2-360M-Inst [8]	+2.3	-1.7	+1.3	+0.3	-0.3	-3.0	**-0.18**
danube3-500M-chat [11]	+0.0	+0.0	+0.0	-0.3	+0.7	+3.0	**+0.57**
Qwen2.5-0.5B-Inst [9]	+3.4	+0.4	+0.0	-1.4	+2.0	+0.0	**+0.73**
Qwen2.5-Coder-0.5B [9]	+0.0	+0.7	+1.6	+0.0	+0.3	-0.3	**+0.38**
LFM2-700M [10]	+1.4	-0.6	-0.3	+0.0	+0.4	-2.4	**-0.25**
Qwen3-0.6B [9]	+1.7	-1.7	+0.0	-1.0	+0.6	+1.4	**+0.17**

Claim boundaries. First, LocalAgent's deployment result is four-thread server-CPU decode; App. A's Galaxy S22 and Tab A8 measurements are for needle2's 2-bit vendor engine and are not directly comparable, so LocalAgent phone latency, memory, energy and thermals remain unmeasured. ToolBench is first action, ToolSandbox has 17 rows, MCP-Atlas lacks schemas, BFCL retains 204 of 1,510 candidates, AndroidControl loses 62.1% under text-view deduplication, and the text-only GUI suites are floor-dominated. Table 1 therefore reports dataset-derived suite scores under our harness, not ten official benchmark scores. Most interventions are one checkpoint; the band is one configuration's threshold, not a per-model interval, and suite means are unweighted. Cross-family rows co-vary corpus, tokenizer, architecture, context, rate and budget; the shipped recipe is unmatched, and literal audits exclude direct but not semantic or upstream overlap. Task completion exists only in our simulator (§5, App. E), with every model at zero on the fully held-out composition.

References

[1] Liu, W. et al. ToolACE: Winning the Points of LLM Function Calling. arXiv:2409.00920, 2024.

[2] Li, W. et al. On the Effects of Data Scale on UI Control Agents (AndroidControl). NeurIPS D&B, arXiv:2406.03679, 2024.

[3] Deng, X. et al. Mind2Web: Towards a Generalist Agent for the Web. NeurIPS, 2023.

[4] Zhang, Z. et al. AgentCPM-GUI: Building Mobile-Use Agents with Reinforcement Fine-Tuning. arXiv:2506.01391, 2025.

[5] Wang, X. et al. OpenCUA: Open Foundations for Computer-Use Agents (AgentNet dataset). arXiv:2508.09123, 2025.

[6] Liu, Z. et al. APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets (xLAM-60k). NeurIPS D&B, arXiv:2406.18518, 2024.

[7] Hu, E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022.

[8] Allal, L. B. et al. SmolLM2: When Smol Goes Big. arXiv:2502.02737, 2025.

[9] Qwen Team. Qwen2.5 (arXiv:2412.15115, 2024) and Qwen3 (arXiv:2505.09388, 2025) Technical Reports.

[10] Liquid AI. LFM2 Technical Report. arXiv:2511.23404, 2025.

[11] Pfeiffer, P. et al. H2O-Danube3 Technical Report. arXiv:2407.09276, 2024.

[12] Neyshabur, B., Sedghi, H., Zhang, C. What is Being Transferred in Transfer Learning? NeurIPS, 2020.

[13] Erdogan, L. E. et al. TinyAgent: Function Calling at the Edge. EMNLP (Demo), arXiv:2409.00608, 2024.

[14] Belcak, P. et al. Small Language Models are the Future of Agentic AI. arXiv:2506.02153, 2025.

[15] Shen, W. et al. Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324, 2024.

[16] Żywot, A. et al. Can Small Agents Collaborate to Beat a Single Large Language Model? arXiv:2601.11327, 2026.

[17] Qin, Y. et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs (ToolBench). ICLR, arXiv:2307.16789, 2024.

[18] Chen, T., Goodfellow, I., Shlens, J. Net2Net: Accelerating Learning via Knowledge Transfer. ICLR, arXiv:1511.05641, 2016.

[19] Chen, C. et al. bert2BERT: Towards Reusable Pretrained Language Models. ACL, arXiv:2110.07143, 2022.

[20] Wang, P. et al. Learning to Grow Pretrained Models for Efficient Transformer Training (LiGO). ICLR, arXiv:2303.00980, 2023.

[21] Samragh, M. et al. Scaling Smart: Accelerating LLM Pre-training with Small Model Initialization (HyperCloning). arXiv:2409.12903, 2024.

[22] Samragh, M. et al. Weight Subcloning: Direct Initialization of Transformers Using Larger Pretrained Ones. arXiv:2312.09299, 2023.

[23] Kim, Y., Rush, A. M. Sequence-Level Knowledge Distillation. EMNLP, arXiv:1606.07947, 2016.

[24] Boizard, N. et al. Towards Cross-Tokenizer Distillation: the Universal Logit Distillation Loss. arXiv:2402.12030, 2024.

[25] Minixhofer, B., Paischer, F., Rekabsaz, N. WECHSEL: Effective Initialization of Subword Embeddings for Cross-lingual Transfer. NAACL, arXiv:2112.06598, 2022.

[26] Liu, Z. et al. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. ICML, arXiv:2402.14905, 2024.

[27] Chen, W., Li, Z. Octopus v2: On-device Language Model for Super Agent. arXiv:2404.01744, 2024.

[28] Lin, Q. et al. Hammer: Robust Function-Calling for On-Device Language Models via Function Masking. ICLR, arXiv:2410.04587, 2025.

[29] Repantis, V. et al. How Many Tools Should an LLM Agent See? A Chance-Corrected Answer. arXiv:2605.24660, 2026.

[30] Zhang, Q. et al. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. ICLR, arXiv:2303.10512, 2023.

[31] Ben Zaken, E., Ravfogel, S., Goldberg, Y. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked LMs. ACL, arXiv:2106.10199, 2022.

[32] Rücklé, A. et al. AdapterDrop: On the Efficiency of Adapters in Transformers. EMNLP, arXiv:2010.11918, 2021.

[33] Lee, Y. et al. Surgical Fine-Tuning Improves Adaptation to Distribution Shifts. ICLR, arXiv:2210.11466, 2023.

[34] Panigrahi, A., Saunshi, N., Zhao, H., Arora, S. Task-Specific Skill Localization in Fine-tuned Language Models. ICML, arXiv:2302.06600, 2023.

[35] Hase, P., Bansal, M., Kim, B., Ghandeharioun, A. Does Localization Inform Editing? Surprising Differences in Causality-Based Localization vs. Knowledge Editing. NeurIPS, arXiv:2301.04213, 2023.

[36] IBM Granite Team. Granite 4.0 Language Models. IBM Research, 2025.

[37] Ndubuaku, H. et al. Needle 2: A 45M-Parameter Foundation Tool-Calling Model for Tiny Devices. Cactus Compute model and software release, 2026.

[38] Patil, S. G., Mao, H., Ji, C. C.-J., Yan, F., Suresh, V., Stoica, I., Gonzalez, J. E. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. ICML, PMLR 267, 2025.

[39] Lu, J., Holleis, T., Zhang, Y. et al. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. arXiv:2408.04682, 2024.

[40] Scale AI. MCP Atlas: Benchmarking Tool Use over Real MCP Servers. Technical report; dataset ScaleAI/MCP-Atlas (Hugging Face); code scaleapi/mcp-atlas (GitHub), 2025.

[41] Google. Mobile Actions: Android system-tool function-calling traces, the FunctionGemma-270M training and evaluation set. Dataset google/mobile-actions (Hugging Face), 2025.

[42] Agent Ark. Toucan-1.5M: 1.5M tool-agent trajectories synthesized from real-world MCP environments. arXiv:2510.01179; dataset Agent-Ark/Toucan-1.5M (Hugging Face), 2025.

[43] Nous Research and Lambda. Hermes Agent Reasoning Traces: multi-turn tool-calling trajectories with executed tool results. Dataset lambda/hermes-agent-reasoning-traces (Hugging Face), 2026.

[44] Glaive AI. Glaive Function Calling v2. Dataset glaiveai/glaive-function-calling-v2 (Hugging Face), 2023.

[45] r0b0tlab. qwen3.8-max-glm5.2-kimi-k3-distillation: pooled frontier-model distillation traces. Hugging Face dataset, 2026.

[46] Yu, M. et al. The Super Weight in Large Language Models. arXiv:2411.07191, 2024.

[47] Biderman, S. et al. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. ICML, 2023.

[48] Gu, A., Dao, T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. COLM, 2024.

[49] Radford, A. et al. Language Models are Unsupervised Multitask Learners (GPT-2). OpenAI, 2019; Sanh, V. et al. DistilBERT distillation applied as DistilGPT-2, 2019.

[50] Ding, N. et al. Enhancing Chat Language Models by Scaling High-quality Instructional Conversations (UltraChat). EMNLP, 2023.

[51] Bai, Y. et al. Training a Helpful and Harmless Assistant with RLHF (hh-rlhf). arXiv:2204.05862, 2022.

[52] Schick, T. et al. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS, 2023.

[53] Yao, S. et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR, 2023.

[54] Patil, S. G. et al. Gorilla: Large Language Model Connected with Massive APIs. NeurIPS, 2024.

[55] Chen, M. et al. Facilitating Multi-turn Function Calling for LLMs via Compositional Instruction Tuning (BUTTON). ICLR, arXiv:2410.12952, 2025.

[56] Chai, H. et al. PARL-MT: Learning to Call Functions in Multi-Turn Conversation with Progress Awareness. arXiv:2509.23206, 2025.

[57] Yao, S., Shinn, N., Razavi, P., Narasimhan, K. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045, 2024.

[58] Yenugula, R. S. D. P. Wiring Beats Blending: What Transfers Between Transformer Sizes—and What Doesn't. arXiv:2608.02829, 2026.

A Corpus, suites, and further limitations

These appendices expand the main text's analyses; every headline number is in the body.

Each suite's source artifact, scored split, training-corpus use, and reference:

Table 5: Provenance and scope of the ten public evaluation suites. The first three dataset-author call splits and the seven study projections / filtered views are visually separated in Table 1; none is presented as an official leaderboard score. Rows = evaluation-pool size after normalisation; appendix tables score the full pool where their arms have full-pool receipts and the first-200 protocol otherwise. Suites marked *never* contribute no training rows; the three union sources (§3) and three §4 imports use the source's own train split, deduplicated against the evaluation pool at the rendered-prompt level.

Suite	Domain	Source artifact	Split scored	Rows	Train split used
AndroidControl [2]	mobile GUI	HF OfficerChul/Android-Control-84k (text-only mirror)	held-out, action-balanced; dedup audit keeps 343/904	904	union corpus (§3)
Mind2Web [3]	web GUI	HF osunlp/Mind2Web	text-only view held out from pinned train file	252	union corpus (§3)
AgentNet [5]	desktop GUI	HF xlangai/AgentNet (OpenCUA)	text-only view, evaluation-only	12,482	never
ToolACE [1]	function calling	HF Team-ACE/ToolACE @ 6bda777c	held-out shard of the pinned release	949	union corpus (§3)
xLAM [6]	function calling	HF Salesforce/xlam-function-calling-60k	dataset authors' test shard	2,941	wide import (§4)
BFCL [38]	function calling	HF gorilla-llm/Berkeley-Function-Calling-Leaderboard	contract-compatible subset, 204/1,510	204	never
ToolBench [17]	function calling	OpenBMB/ToolBench (ToolLLM) G1+G2+G3 DFS split	first-action projection, evaluation-only	687	never
ToolSandbox [39]	function calling	GitHub apple/ToolSandbox scenarios	single-tool, single-turn projection	17	never
MCP-Atlas [40]	tool use (MCP)	HF ScaleAI/MCP-Atlas	first-action projection (495/500 open with a call)	495	never
Mobile-Actions [41]	mobile system tools	HF google/mobile-actions	dataset authors' eval split	961	wide3 import (§4)

Asset licences. ToolSandbox is used under the Apple Sample Code License; MCP-Atlas is CC-BY-4.0. The pretraining-source table below records the remaining corpus licences.

The AndroidControl deduplication. The text-only mirror replaces screenshots with a fixed prefix plus the instruction, so episodes differing only in screen state collapse into byte-identical user turns; a test row is dropped when its user turn occurs in the training split. The 200-row protocol lost a quarter of the sample; the full 904-row pool loses 561 rows (62.1%), contamination larger than the sample suggested. Among surviving twins, one shares its gold call verbatim across splits (pure memorisation credit) while another's golds differ only in capitalisation, charging the memoriser an exact-match error — why exact match falls harder than name match once twins are removed.

ToolSandbox, what the projection shows. Our catalog-conditioned models emit fully parseable calls yet select the gold tool 0 of 17 times, indistinguishable from the 3.0% chance-in-catalog floor. Ranking all 33 candidates by length-normalised sequence logprob at an 8,192-token window hides no weak preference: mean gold rank 18.4/18.9 against a uniform 17.0 (z = +0.6/+0.8), reciprocal rank 0.114/0.099 against 0.124, top-five 2 of 17 against 2.6 expected. A disclosed control closes the loop: adding 1,066 conversations synthesised from the same 33 public schemas (no test scenario read) still selects gold 0 of 17, every other suite inside its spread. The deficit is not name coverage.

ToolBench's floor is the most informative of the function-calling suites: 5.9 candidates per task against 2.1–3.4 for the others puts chance within the row's own catalog at 22.9% rather than the 45.8–61.4% those suites leave — the most headroom and the sharpest test of catalog reading; the novel-inventory suites leave lower floors still (MCP-Atlas 6.7%, ToolSandbox 3.0%).

needle2, and what is and is not claimed. Cactus needle2 [37], the one released model in our size class (a 45M tool-calling specialist), is a 2-bit engine with grammar-constrained decoding, a top-5 tool-retrieval head and a confidence gate that abstains and escalates; scored through its own engine, catalogs above five tools pass through the retrieval head and abstentions count as unparsed, folding coverage and correctness together. As precision-when-answering (200-row audit): 90–100% on the four function-calling suites at 27–37% coverage; its one lead, ToolSandbox (64.7 vs our pretrained 0/17), may reflect better compatibility with its pretrained device-assistant distribution or retrieval; this study does not isolate the cause. Its tok/s cell is the engine's self-reported decode rate beside f16 models under the 512-prefill/64-decode protocol — not apples-to-apples — and its android build brackets a 5.8× hardware range: a Galaxy S22 sustains 643–654 tok/s at 23 MB peak RAM (top of the maker's 300–700 claim), a sub-$200 Galaxy Tab A8 112 tok/s, below the claimed floor though interactive — this study's only on-device measurements.

needle2, fine-tuned. The tuned variant trains through the vendor's own trainer, crossing the engine boundary; it gains open-catalog calling like every model under the recipe (xLAM 34.3→89.4 at full pools) but, alone among them, loses device control (AndroidControl 37.9→23.3, where all eleven open baselines gain, +17.4 to +78.0). The vendor's trainer does not update the confidence head: the tuned row reports no calibrated confidence and does not share the abstention behaviour above.

The pretraining streams and the midtrain and SFT conversation splits are tabulated below; one SFT harness consumes them identically for every model, so a row in Table 1 or 2 differs from its neighbour only in the splits its recipe names.

Table 6: Construction of the two pretraining streams, from their shard manifests. *pt-big* is the curated stream behind the Table 1 ladder; *pool-general* is the open-dataset stream behind the staged backbone of App. C, assembled entirely from openly released dialogue and distillation corpora. Both are packed once, with near-deduplication and a rendered-prompt evaluation denylist, into 2,048-token rows under our models' 16k vocabulary; the Table 1 budgets consume the same stream for 1,599 or 16,000 steps, so budget comparisons vary optimisation, never the text. Per-document licence counts ship in the manifests.

Source	documents	share	licence
pt-big — the curated stream (Table 1 ladder)			
fineweb-edu (deduplicated) — educational web text	228,233	62%	ODC-BY-1.0
cosmopedia-v2 — synthetic encyclopedia and textbooks	87,759	24%	Apache-2.0
permissive-python — permissively licensed code	51,788	14%	MIT/BSD/Apache mix
total	**367,780**		
pool-general — the open-dataset stream (staged backbone)			
UltraChat — open multi-turn dialogue	1,713,855	76%	MIT
hh-rlhf — helpfulness dialogue	325,905	15%	MIT
kimi-k3 traces [45] — frontier distillation traces	85,857	4%	open release
qwen3.8-max traces [45] — frontier distillation traces	73,740	3%	open release
r0b0t general [45] — pooled distillation text	45,011	2%	open release
manus traces [45] — agent-style traces	185	0%	open release
total	**2,244,553**		

One row set for every model. BFCL writes Python-flavoured schema types, and its gold arguments do not always satisfy their own declared schema; a model that builds its own catalog never notices, while a catalog-conditioned model's contract refuses such rows and emits nothing, which the scorer counts wrong — under the then-200-row protocol our models saw 94 of the 200 sampled rows and the others saw all of them. The normaliser now coerces the schemas and keeps only rows the contract will render (204 of 1,510 candidates survive), so both groups are scored on identical data; the correction moved LocalAgent-96M from 27.5% to 51.5% on BFCL and its parse rate from 43.5% to 91.2%. The filter applies identically to every model; like ToolBench, this subset is not an official BFCL score and should not be quoted as one.

The replicate noise floor. The per-suite spread every claim is read against is the range across five runs of one unchanged 10.5M configuration, four training seeds between them, re-scored on the full pools; each suite's spread is additionally floored at the value of one row of its pool: a 17-row suite cannot resolve a finer difference whatever the replicates do. The band is Table 2's last row.

Table 7: All splits pass the trainer contract (schema whitelist, gold-argument conformance, duplicate-name and marker rejection, real-tokenizer length) and are deduplicated against the evaluation pools at the rendered-prompt level. Episode rows are one decision each, their user turn byte-identical to the runtime's observation; the 4× corpus is newly generated tasks at a higher error-injection rate, not the base corpus repeated, and its evaluation split is unchanged. Together the three tables cover every token any stage consumes.

Split	Role	rows	Source and filter
merged-v2 union	midtrain agent mix; the shared SFT corpus	16,300	3 public corpora + 9.9% synthetic multi-turn conversations; not simulator decisions
distill2 train-clean	the union, teacher-relabelled (§4)	40,812	teacher call kept only on gold agreement
xLAM split	§4 data-import arm	18,000	Salesforce xlam-function-calling-60k [6]
Mobile-Actions train	the ladder's device split	8,692	official train split [41]
ToolBench train	import arm, 70% eval-name overlap	6,000	from the 46,886-row ToolLLaMA split [17]
Toucan calls	diverse real-MCP import	5,911	single-turn originals [42]
r0b0t calls	staged midtrain agentic split	4,845	tool rows of [45]
device-assistant	stem-overlap control (0% exact)	2,251	API-Bank train + ToolTalk scenarios
ToolSandbox schema-synth	same-inventory arm, disclosed	1,066	from the 33 public schemas; no test scenario read
episode decisions (base)	agentic arms' episode split (App. E)	9,072	1,400 generated tasks, 7 families balanced, gold plans mean 4.9 / max 10 steps, 1.45M tokens; 2,312 recovery and 200 abstention decisions
episode decisions (4×)	the 4× arms' split: new tasks, error injection 0.45 vs 0.35	38,905	5,600 generated tasks, 7 families balanced, gold plans mean 4.9 / max 10 steps, 6.27M tokens; 11,797 recovery and 800 abstention decisions

Whether that floor does statistical work, and where it does not. It is a range over replicates, not a confidence interval, and Table 2 reads its 21 interventions against every readable suite cell, so multiplicity is a fair concern. Under a two-proportion z test on each cell's own pool sizes, the band is *stricter* than a Bonferroni-corrected test on most cells but not on all. Of 210 comparisons the band bolds 27; 21 clear Bonferroni at $\alpha = .05/210$ ($z > 3.67$) and 6 do not: 4 on MCP-A (smallest $z = 2.60$, $1.6 \rightarrow 4.4$); 1 on ToolBench (smallest $z = 3.60$, $31.9 \rightarrow 41.2$); 1 on ToolACE (smallest $z = 3.52$, $51.3 \rightarrow 59.3$). The MCP-Atlas failures sit where a base rate is near zero and the pool is small, so two or three points is large against the replicate band and small against the interval those rows admit; they are movement off a floor, not a measured effect. The remaining failures are near-threshold gains. All are bolded by our rule and should not be read as established. Three caveats stand: the band is measured on one 10.5M configuration and assumed to carry — checked at 96M with three seeds of the Table 1 arm, it holds on 8 of 10 suites and is exceeded on AgentNet (28.3 against 25.3), MCP-A (4.8 against 2.2), so the bolded MCP-Atlas gains of two to three points sit inside the seed range measured at the very scale they are claimed at; it prices training-run variance while the z test prices only sampling; and on a small pool the band can be narrower than the interval that pool admits — the exceptions above. Where they disagree we obey the band, except those cells. The z test is also unpaired where McNemar would have more power; the weaker test is conservative.

Which noise source binds, per suite. The band prices training-run variance, not how precisely a suite can measure a rate — its pool size fixes that; below, the 95% Wilson interval for a cell on each suite is set against the replicate band. On the large pools the band is the larger and seeds bind; on the small ones the pool binds, and no number of training runs would resolve a finer difference.

What the cost column measures, and what it does not. One server CPU at four threads, fp16, a 512-token prompt and a 64-token answer, repeated for every row of Table 1 under one protocol. Two quantities come out of that run: decode throughput, which the table reports, and prefill, time to first token — 22.3 ms for the 10.5M hybrid, at 22,970 tokens/s. That rate is what a long catalog costs: the 33-schema ToolSandbox prompt, roughly seven times this one, is charged accordingly — a catalog-in-prompt cost paid before the first token. Not measured: resident memory, energy per decision, and the motivating hardware — no number here was taken on a phone, Raspberry Pi or

embedded board — and the order-of-magnitude decode gap over a 0.5B model is a statement about this machine under this protocol, not a device measurement.

B Weight transfer in detail

How each model family is fine-tuned. Open baselines take full-parameter updates: AdamW 2e-5 with warmup-decay, batch 8 at maximum length 1,024, 600 steps on the union corpus, gradient steps skipped on non-finite norms. The adapter alternative — LoRA [7] at rank 16 (alpha 32, dropout 0.05) on every attention and feed-forward projection, AdamW at 1e-4 — sums lower over the block (table below), so the gate selects full parameters; 1e-4 is a normal adapter rate, roughly five times a normal full-parameter one: each method runs at its own rate. One scoring exception: Granite-4.0-H is *scored* at batch 4, its mamba2 path materialising a six-index tensor whose 32-row batch requests 144 GiB. The objective is token-level cross-entropy with prompt tokens masked (only the answer span trains) under each model's own chat template. Declared exceptions: the Mamba baselines take the same full-parameter updates at batch 2, maximum length 768, for the torch-native scan's memory, steps and rate unchanged (the released ladder in App. C keeps their earlier in/x/dt adapter exception); needle2 uses the vendor's trainer — the one engine-boundary crossing — its confidence head not updated (App. A). Our models take full-parameter SFT: length 2,048 under the openai-full-catalog contract, AdamW 2e-4 with WSD, micro-batch 8, accumulation 4 (effective batch 32), 900 steps on two-source recipes, 1,500 on five-source (step count saturated; App. C). The weight-transfer arms below inherit these settings, varying only initialisation.

Table 8: Which configuration produced each row family. The open-block row follows the method gate; the Mamba and Granite-4.0-H exceptions are as stated above. Steps are optimisation steps; batch is micro-batch × max length. Open episode arms (SmolLM2-135M, Qwen3-0.6B): 2,329 episode decisions sampled at seed 2026 into the 16,300-row union (12.5%); one training seed per arm; control = the same model's Table 1 checkpoint; no episode row exceeds the 1,024-token window (longest 731). merged-v2 composition: ToolACE 6,000, AndroidControl 6,000 and Mind2Web 2,694 public rows plus 1,606 author-synthetic multi-turn conversation rows (9.9%). Those rows serialize planning and tool chaining, but do not contain the simulator's goal/state/last-error decision observation. App. E's episode rows do; the two sources are not interchangeable. The generator ships with the supplementary code.

Rows	Update type	Optimiser	Rate	Steps	Batch	Corpus	Script
Open block (11 models, Table 1)	full parameters	AdamW	2e-5	600	8 × 1,024	union (16,300)	finetune_public.py
Mamba pair & Granite-4.0-H	as above	as above	as above	600	2 × 768	union	finetune_public.py
LoRA reference block (App. B)	LoRA r=16	AdamW	1e-4	600	8 × 1,024	union	finetune_public.py
needle2	vendor trainer	vendor defaults	—	—	—	union	vendor CLI
This-work Table 1 rows	full parameters	AdamW	2e-4, WSD	1,500	8 × 2,048 (accum 4)	five-source	localagent train sft
This-work baserecipe (App. B)	full parameters	AdamW	1e-4	600	8 × 2,048	union	localagent train sft
Agentic arms (App. E)	full parameters	AdamW	2e-4, WSD	1,500	8 × 2,048 (accum 4)	five-source + episodes	localagent train sft
Open episode arms (App. E)	full parameters	AdamW	2e-5	600	8 × 1,024	union + episodes 12.5%	finetune_public.py

Fairness of the shared recipe to the open models. Our models train full-parameter because this trainer has no adapter path, and the method gate has since selected full parameters for the open block, so update type is matched across Table 1; unmatched are learning rate (2e-4 under WSD against the block's 2e-5) and the budget; the table below prices the choice. Re-fine-tuning three block-spanning models with full parameters at three learning rates on the same corpus and budget lands the two methods within two points on both larger models at a rate chosen for full updates; the adapter is not carrying the comparison, and one method serves every open row.

Fine-tuning matched, pretraining each party's own. The matched control gives our base the open block's union corpus, 600 steps, batch 8 and AdamW 1e-4. That stage is the block's LoRA-era recipe; Table 1's block has since moved to full parameters at 2e-5, so this control matches its update type and differs in rate. Re-running at the block's own 2e-5 gives 95.3M author 39.3→43.9, projection 45.6→40.7; 42.8M author 41.1→43.1, projection 39.0→40.3. The effect is mixed across scope and size; the rate-asymmetry table below reports it rather than pooling it. We report no combined rank: LocalAgent-96M scores 39.3/45.6 on the dataset-author / study-projection groups while needle2 scores 74.0/30.7, so the apparent tie under the former pooled mean was an artefact of mixing scopes. Per-suite values and all three sizes are in the table below. This is generous to us: midtrain already consumed the union corpus, and our 2,048-token catalog is untruncated where open harnesses use 1,024.

Table 9: Is the open block held back by the earlier adapter recipe? The same corpus and the same 600-step budget, varying only the method and its learning rate, on three models spanning the block. At a learning rate chosen for it, full-parameter fine-tuning is within 2.7 points of the earlier LoRA recipe in both scope groups for the larger models, so the adapter is not what decides the comparison; 1e-4 is a normal LoRA rate and roughly five times a normal full-parameter one, which is why the full arm collapses there without ever losing its output format (parse rates stay at 99–100%). The exception is the smallest model, which prefers full parameters at the high rate in both groups. We score every open row with full parameters at 2e-5, which sums higher within both scope groups over the block (the switch is a single gate, never per-model). Each model reports dataset-author / study-projection descriptive means for function-name match (%), never one combined score.

Fine-tuning method	SmolLM2-135M		LFM2-350M		Qwen3-0.6B	
	author	projection	author	projection	author	projection
LoRA r=16, 1e-4 (earlier reference)	68.6	44.4	94.7	58.8	96.3	57.8
full parameters, 1e-4	79.9	52.9	81.6	50.6	86.8	47.7
full parameters, 2e-5	73.7	50.8	94.1	56.1	96.4	58.0
full parameters, 1e-5	65.0	46.3	95.1	56.2	96.8	59.3

Table 10: Learning-rate asymmetry in the matched LocalAgent control. Both rows use full parameters, the same union corpus, 600 steps and the same initial checkpoint; only AdamW's rate changes. Values are the two declared scope means (%), not a ten-suite leaderboard average.

Model	Rate	Author splits	Study projections
43M	1e-4	41.1	39.0
43M	2e-5	43.1	40.3
96M	1e-4	39.3	45.6
96M	2e-5	43.9	40.7

Table 11: Our pretrained models under the open baselines' data and step budget instead of the LocalAgent shipped recipe. Pretraining, midtrain and prompt contract are identical to the Table 1 LocalAgent row; the SFT stage changes to the union corpus, 600 steps, batch 8 and AdamW at 1e-4. The no-SFT row is the shared initial checkpoint, and the union-at-our-budget row separates corpus from optimisation where available. Two asymmetries remain explicit: this trainer has no adapter path, so the matched arm is full-parameter, and maximum length stays 2,048 because at 1,024 the catalog-conditioned contract refuses to render at all (a row needs 1,042 tokens and a catalog is not truncatable), which is itself the asymmetry — the baselines fit in 1,024 only because their harness truncates the prompt. Function-name match (%).

Fine-tuning recipe	AndroidCtrl	Mind2Web	AgentNet	ToolACE	xLAM	BFCL	ToolBench	ToolSB	MCP-A	MobAct
LocalAgent-11M										
five-source recipe (Table 1)	67.1	80.2	55.5	33.4	73.1	49.0	28.2	0.0	4.2	80.6
the baselines' recipe	52.1	80.2	59.1	35.7	43.8	44.6	12.7	0.0	0.8	22.2
the baselines' corpus at our budget	78.7	80.6	53.5	34.7	38.6	42.2	15.7	5.9	0.8	25.5
no SFT (the shared initialisation)	59.0	19.4	36.1	29.3	39.9	37.3	12.2	0.0	1.2	21.6
change	**-15.0**	+0.0	+3.6	+2.3	**-29.3**	-4.4	**-15.6**	+0.0	**-3.4**	**-58.5**
LocalAgent-43M										
five-source recipe (Table 1)	65.7	80.2	38.0	52.5	91.3	69.1	41.0	0.0	2.0	81.2
the baselines' recipe	74.2	80.2	37.8	41.2	52.2	52.9	24.5	0.0	3.6	29.8
the baselines' corpus at our budget	75.8	87.3	8.1	42.6	47.8	51.0	21.3	0.0	0.8	22.7
no SFT (the shared initialisation)	65.2	74.6	50.1	35.2	48.3	53.9	21.5	0.0	4.8	27.0
change	+8.5	+0.0	-0.2	**-11.3**	**-39.1**	-16.2	**-16.6**	+0.0	+1.6	**-51.4**
LocalAgent-96M										
five-source recipe (Table 1)	67.4	80.2	47.3	58.5	90.1	66.2	41.2	0.0	2.6	81.9
the baselines' recipe	79.2	80.2	68.7	43.6	56.7	64.2	23.6	0.0	3.0	17.5
the baselines' corpus at our budget	79.9	86.1	12.9	52.9	59.9	57.8	15.9	0.0	4.2	41.4
no SFT (the shared initialisation)	67.0	75.4	42.8	40.4	54.0	63.2	22.0	0.0	5.5	14.8
change	+11.8	+0.0	+21.5	**-14.9**	**-33.4**	-2.0	**-17.6**	+0.0	+0.4	**-64.4**

Where the five-source points sit. Every readable 43M/96M difference is on a function-calling suite whose split the five-source stage adds; uncovered suites do not move. The apparent 96M device gains are inside spread (AndroidControl +11.8 versus 14.8; AgentNet +21.5 versus 25.3, with parse rate 67.6→98.6), so we claim none. Mobile-Actions shows naming rather than formatting: the matched arm parses 93.4% versus 82.5% yet names the tool 17.5% versus 81.9%. On the three largest drops, union/five-source name coverage is 14.3/100, 1.9/95.0 and 0.0/70.1%; corpus replacement buys +9.5/+16.4/+12.4 while extra budget buys only −2.6 to +2.5. ToolACE's −14.9 at nearly fixed coverage likely reflects teacher relabelling; AgentNet remains floor-dominated.

The least-written third recovers most of the gain. Placement is documented as forgiving [30–32], shift-dependent [33], a poor guide to intervention [34, 35]; the band test: if ability lived where the update concentrates, the top third should recover the all-layer recipe and the bottom third should not. The opposite holds: over three seeds per band on two models, the bottom band recovers 85% of the gain against the top's 65%, beating it by 13–16 points on AndroidControl with no seed-range overlap; a random third lands between. Update magnitude records where optimisation wrote, not where ability lives; scale relocates the site without changing it (Figure 2b). Depth pruning agrees: cut to 10 of 28 layers, Qwen3-0.6B keeps any function calling only under the *least-written* policy of four (ToolACE 10.0, xLAM 16.2, BFCL 8.8 against 0.0–3.4), though none clears chance: depth alone only chooses how the model breaks.

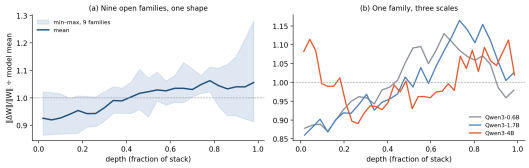

Figure 2: Each layer's $\|\Delta W\|/\|W\|$ divided by its model's mean. (a) Mean and min–max band over nine open families across three architecture classes: below the model's own mean through the first third and above it thereafter for seven families; two SmolLM2 variants are exceptions. (b) Qwen3 at three scales: the band moves later and turns bimodal at 4B.

When shapes match and no projection is needed, parameter share is nearly inverted as a predictor of value: the embedding table, the largest region in a 16k-vocabulary model at this scale (59.8% of all weights), is worth less alone than the 40.2% that is everything else. The computation in the blocks transfers, not the lookup table in front of them: a vocabulary change, which forces a fresh embedding, costs far less than its parameter count suggests.

Matched optimisation traces after cross-shape initialisation. Figure 3 reads the actual 16,000-step matched runs. Initial validation loss spans 9.777–9.878; endpoints are random 2.462, skeleton 2.407, feed-forward 2.437, embedding-only 2.414. Projection finishes 0.025–0.055 below random, so optimisation does not fail, but no arm clears an open-catalog replicate spread (Table 2): lower language-model loss is not behavioural transfer. Same-shape adoption bypasses pretraining and is tested by the region controls.

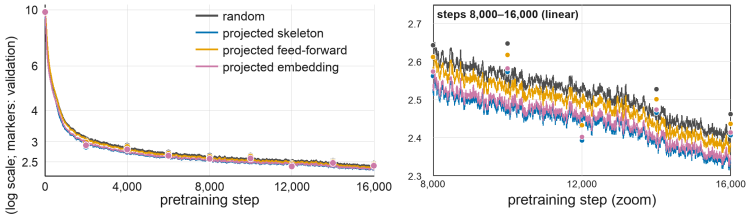

Figure 3: Matched 96M pretraining after random or cross-shape projected initialisation: smoothed per-step training loss; markers show validation every 2,000 steps.

What the cross-shape result does and does not cover. The negative result is specific to coordinate-preserving projections — leading-block truncation, head-slicing and this appendix's size-ratio width crops — which carry nothing across an architecture boundary that random initialisation does not already give on the downstream suites. Untested methods that could each change the answer: a learned

projection or growth operator in the LiGO style [20], hidden-state or feature distillation, logit distillation, alignment of layers before mapping, low-rank factorisation of the donor, function-preserving reductions, structured pruning with re-training, and tokenizer-aware embedding conversion; only the last two have partial evidence, from the depth-pruning arms above. With the same-shape arms transferring strongly, the supported statement is that naive cross-shape slicing fails, not that cross-shape transfer is impossible.

What transfers is not evenly distributed. Section 4 says the layers fine-tuning writes hardest are not the layers worth adapting, a statement about update magnitude within one model, not that no region is privileged; the same-shape transfer arms say the opposite. Averaged over all seven open-catalog suites against random initialisation, adopting the first half of the blocks gains 0.1 points and the second half 13.0; attention gains 14.7 and feed-forward 0.6, against 21.0 for the whole backbone. Two of the four regions, the later blocks and the attention projections, carry nearly everything. Whether that is where the ability sits, or only where a sibling's weights happen to be reusable, these arms do not separate and we do not claim; they establish that the transferable content is concentrated, positionally and structurally rather than in proportion to parameter count.

C General capability, absolute scores

Six suites span what an agent recipe could plausibly damage. The available receipts compare released weights with the earlier LoRA reference checkpoint (`runs/lora/*`), not Table 1's current full-parameter stage: commonsense completion (HellaSwag), coreference (Winogrande), grade-school and elementary science (ARC-e, ARC-c, OpenBookQA) and broad knowledge (MMLU). Scoring is length-normalised continuation likelihood over 300 rows per suite, identical rows for the released and the adapted weights, so the comparison is free of prompt-format and generation effects. The headline is unchanged: across the eleven baselines the mean change spans -1.8 to $+0.7$ points, inside the ± 2.9-point sampling error, and the one family-level regression is Granite-4.0's 5.7–7.3-point loss on ARC-e.

Table 12: Five open corpora as the pretraining stage, packed into the same shard layout and run through the identical chain at the same 1,599-step budget: every arm consumes the same 209.7M tokens, the corpus size below is what the budget samples from, not what it reads, and implied epochs vary by two orders of magnitude. Function-name match (%) after the shared post-training; each pretraining stage took 20–21 minutes on one otherwise idle H100 NVL, sequentially. A packing fault made all five arms consume the identical original token stream (retraction below the table): read these rows as replicates, not a corpus comparison.

Pretraining corpus	docs	tokens (M)	chars/tok	pretrain (min)	ToolACE	xLAM	BFCL	ToolBench	ToolSB	MCP-A	MobAct
Qwen3.8-Max distillation (50k)	141k	39	3.28	21	32.1	37.9	46.1	15.7	0.0	1.8	14.4
Manus multi-teacher distillation	125k	11	4.39	21	35.5	39.7	45.6	11.8	0.0	3.2	19.3
Anthropic HH-RLHF	361k	94	3.83	21	35.0	38.9	44.1	13.7	0.0	4.8	8.7
Qwen3.8/GLM5.2/Kimi-K3 distillation	367k	156	2.94	20	35.8	41.8	47.1	8.0	0.0	3.4	13.3
UltraChat-200k	2671k	520	4.37	20	35.2	42.0	45.6	14.0	0.0	2.4	11.2

Retraction, and the corrected arm. The five-corpus pretraining comparison was retracted: shard preparation copied the reference corpus's packed tokens alongside the new text, so all five arms consumed the original 465M-token corpus — the corpus-identity comparison is void, its arms replicates. A correctly packed fresh-token arm at the same step count reproduces the ToolACE, xLAM and BFCL gains of §4 but gives back 8.1 of ToolBench's 9.0 points (outside its 6.4 spread), trailing on device control only inside wide spreads: epochs are optimisation, not memorisation, ToolBench alone is repetition-keyed, and a one-epoch pair agrees. Two flags: AgentNet collapses for both initialisations, and Mobile-Actions separates toward the skeleton arm (§4) against its wide spread.

The open-agent pool, and what pretraining cannot import. Three Apache-2.0 corpora (Toucan-1.5M [42], Hermes [43], Glaive v2 [44]) ask what open agent data buys in pretraining. Benchmark-embedding sources were rejected; the one adjacency — Toucan and MCP-Atlas both draw on real MCP servers — shares none of MCP-Atlas's 220 catalog names verbatim. Despite hundreds of millions of genuine MCP tool-call characters, MCP-Atlas stays at or below its 6.7% chance floor and neither arm beats the standard mix on any suite. Via SFT, 5,911 Toucan trajectories nudge the *novel-tool* suites within spread while device control pays the capacity price (AndroidControl 67.0→54.1) — the clearest claim the 200-row cap manufactured. The xLAM import does clear

its spread (+26.8, 99.4% name overlap, nothing elsewhere): imports buy *inventory* exactly where names overlap; whether diversity alone buys *generalising catalog skill* is left open.

The staged pipeline: agent data need not enter pretraining at all. Pretrain on general distillation and dialogue text only (a 743M-token pool: distillation traces [45], UltraChat [50], hh-rlhf [51]), concentrating the open agentic data (Toucan [42], r0b0t [45]) in the 700-step midtrain. At 96M, identical two-source SFT gives author/projection scope means of 77.2/44.7 for curated-web pretraining and 79.0/45.0 for staged pretraining; every per-suite difference is inside spread. With five-source SFT the corresponding means are 76.8/43.5 and 78.5/46.7. These are descriptive scope means, not a pooled leaderboard rank; no constrained decode or as-released comparison is used. Doubling SFT steps moves nothing outside spread. At 10.5M and 43M the staged backbone trails its curated twin (outside spread only on ToolBench at 43M), so Table 1 keeps the curated ladder. At 96M the pretraining corpus can be whatever general text is cheapest — where there is model to absorb it.

The vocabulary and context axis. Rebuilt with an LFM-class 65,536-token vocabulary and 8,192 context (127M parameters, almost entirely embedding table), identical chain and corpus, the larger vocabulary *costs* open-catalog accuracy (ToolACE 45.7 against 54.9 at the 16k model, beyond that suite's replicate spread; BFCL down but not beyond it) and 17% of decode throughput; its one large gain, on AgentNet, sits under that suite's 72.9% majority floor. Doubling context to 16,384 rescues none of this: ToolACE and BFCL stay down, decode does not recover, and AgentNet's swing to 33.8 is the floor-dominated instability that keeps the suite suggestive. At this scale the 16k vocabulary is the better spend on every axis measured.

The mixer, priced. Three arms hold chain, corpus, tokenizer, schedule and contract fixed, varying only how many of four blocks are attention (parameters 10,547,072/10,524,544/10,522,496, within 0.23%). Dataset-author / study-projection descriptive means: attention-only 28.3/27.7, hybrid 28.6/34.6, convolution-heavy 25.7/36.1. The mean is a poor summary of what happened. The suites split into two families that move in opposite directions with attention count. On the four function-calling suites the means run 31.1, 29.5, 25.2 from four attention layers down to one, so attention helps; on the three GUI and desktop suites they run 45.6, 62.8, 69.1, so convolution helps, and the largest single movement is AgentNet at +55.9 between the extremes. The hybrid sits between the two on both families rather than leading either, which is a more specific claim than the pooled mean can make and not the one this paper made before. On decode the mixer is worth something rather than nothing: 701 against 541 tokens/s on four CPU threads, a 22.9% difference — not what puts either arm an order of magnitude ahead of a 0.5B open model at 30 tokens/s; that is the parameter budget. The withdrawn version reported 1.7% here, which was two checkpoints of one architecture and measured no mixer at all.

The mixer on episodes. The convheavy config carried a prediction: if attention count carries verbatim copying, the one-attention-layer arm should fall on it. As episode policies all three complete nothing, as single-call checkpoints should; the step profile separates them. The fraction of steps whose call satisfies its schema rises 0.000, 0.395, 0.854 from one attention layer to four at fixed budget, while verbatim copy fidelity stays flat (0.32, 0.41, 0.38). At this size attention carries assembling a legal call from a live state; the copying the config's guard names does not vary with it.

D Three harness faults, and what they cost

Three harness faults changed baseline numbers, each in our favour until fixed. **LFM2-350M**'s separate `chat_template.jinja` was dropped by a pattern-limited snapshot, so the harness fell back to plain concatenation and the model scored 0.1% on ToolACE; it also answers in a Pythonic `[name(arg="value")]` form, now parsed. **Qwen3-0.6B**'s default thinking block consumed the 64-token budget before any call; the non-thinking path moves ToolACE 0.5%→88.1%. **h2o-danube3-500M**'s template raises on system turns; the harness folds system text into the user turn.

Batched decoding. Every model is decoded in batches of 32. For our architecture this is exactly free: rows are grouped by identical token length (RoPE broadcasts one absolute-position vector, so left padding would mis-position short rows), and generations are token-identical to the one-at-a-time path on a 95.3M and a 10.5M checkpoint. For open models it is not free: batch size changes GEMM reduction order, flipping near-tied early tokens — over 710 cells, 204 up, 172 down, mean −0.005,

mean absolute 0.36 (SmolLM2-135M at most 1.3, Pythia-160M up to 9.9 on Mind2Web). It is a protocol choice, applied to the whole open block; no table mixes the two (Granite-4.0-H's batch-4 scoring exception: App. B). Re-scoring an unchanged checkpoint reproduces every suite exactly, so the harness is deterministic under a fixed batch size.

E Episodes: acting from the state rather than from a script

An earlier protocol scored these tasks through wrappers that dictate each step and reported 1.0 — formatting under dictation. The simulator is sound: `_transition` enforces preconditions with twelve error codes and never consults the gold list; `task_complete` is path-independent containment.

The freed harness. The policy sees goal, state and last error; any of the nine device calls from any state; the episode ends when the goal holds or budget is spent. The generated suite: 210 eval tasks, seven families, slot vocabularies partitioned between splits, a quarter starting dirty. Every task replays its gold plan before use; abstention is scored on the action, whose goal holds at step zero.

Bounds. The oracle reaches 100% in 4.9 steps, progressing on only 24% of them; a uniform policy reaches 0.5%.

Table 13: Episode success on the free stateful suite: 210 tasks over seven families, scored on whether the goal state holds when the episode ends rather than on whether each call matched a script. The oracle and the uniform policy bound the metric; the held-out column is a three-goal, twelve-step family that appears in no training corpus. Step columns are fractions of all steps taken, so they describe the episodes a policy did not finish as well as the ones it did. The uniform policy draws each tool's arguments from that tool's own recorded calls and is therefore schema-valid by construction; its column is not a capability.

Policy	Episode	Milestone	Schema ok	State moved	Progressed	Held-out 3-goal
Oracle (replays the gold plan)	100.0	100.0	97.1	97.1	23.5	—
Uniform over the nine device tools	0.5	0.5	100.0	37.0	0.0	—
LocalAgent-11M, Table 1 checkpoint	0.0	0.0	0.5	0.1	0.0	0.0
LocalAgent-43M, Table 1 checkpoint	0.0	0.0	12.0	0.3	0.0	0.0
LocalAgent-96M, Table 1 checkpoint	0.0	0.0	88.4	50.1	0.0	0.0
LocalAgent-11M, agentic arm	20.5	21.4	82.1	59.1	1.6	0.0
LocalAgent-43M, agentic arm	34.3	39.8	87.2	80.6	4.1	0.0
LocalAgent-96M, agentic arm	17.6	20.5	70.6	56.3	1.7	0.0
LocalAgent-11M, agentic arm, 4x corpus	21.4	26.7	78.1	53.7	2.4	0.0
LocalAgent-96M, agentic arm, 4x corpus	25.2	32.6	86.7	76.6	3.4	0.0

What the Table 1 checkpoint does here. LocalAgent-11M, Table 1's smallest model, completes 0.0% of these episodes and spends 17.1 of its 20 steps. The failure is mostly not format (81% of outputs parse into a call) but conditioning on the state. Of 3,600 steps, 2,881 (80.0%) are rejected because the arguments belong to a different tool than the one named, and only 2 (0.1%) change the state at all, against the uniform policy's 37%. It selects a plausible tool name and argument bag from the goal text without reading the state, reproducing shown text on 69% of text-bearing steps: the arguments are copied faithfully into the wrong call.

Paired tests, at the two units they belong to. Over tasks, checkpoints fixed, McNemar's exact test under Bonferroni across all 66 pairs of the LocalAgent grid (27 arm–baseline, 36 among arms, 3 among baselines; the open-family arms follow a different template and budget and are reported separately) finds all arm–baseline pairs significant (worst $p < 10^{-10}$) and 11 of the 36 pairs among the arms. 2 of those 11 are seed replicates of the *same* procedure (smallest $p = 2\times10^{-5}$, 1/20 discordant) — the decisive caution: task-level differences between fixed checkpoints are not evidence the training procedures differ. Only these two procedures have three seeds; the other LocalAgent arms are single-checkpoint estimates. Task-level significance does not order the training procedures once between-seed variance is considered. Single-checkpoint intervals are percentile bootstraps over 210 tasks and speak only for that checkpoint.

What a seed is worth. Only two procedures have three seeds: agenticxl-96m 0.297 over 3 runs, range 7.1 points, agentic-11m 0.236 over 3 runs, range 9.0 points. All other LocalAgent episode cells are single-checkpoint estimates. The replicated procedures' 7.1–9.0-point ranges are comparable to

the roughly 12-point task-bootstrap width. The 6.0-point mean gap is inside the wider seed range and misses the corrected procedure-level test, so the arms are not ordered. Table 1 checkpoints remain zero at every size and seed.

Sparse shares add; dense shares trade. We compare siblings of the Table 1 checkpoints: the midtrain parent, optimiser, budget and five source corpora are fixed, and only the episode decisions are added. The 12.5% arms show no consistent cost across the two groups (the largest group decline is 0.5 points), while the 38–40% arms lower both groups at the measured sizes: 11M base author 62.4 to 64.7, projection 40.6 to 42.4; 43M base author 75.0 to 74.4, projection 42.3 to 45.9; 96M base author 76.8 to 78.2, projection 43.5 to 47.7; 11M 4× author 62.4 to 60.1, projection 40.6 to 39.1; 96M 4× author 76.8 to 73.1, projection 43.5 to 36.1. The only losses outside replicate spread: 96M base on ToolBench, 41.2 to 34.2; 11M 4× on MCP-A, 4.2 to 1.8; 96M 4× on AgentNet, 47.3 to 9.6; 96M 4× on ToolACE, 58.5 to 50.8; 96M 4× on ToolBench, 41.2 to 32.8. Outside spread in the other direction: 43M base on AgentNet, 38.0 to 70.2. The trade therefore follows episode share rather than a single pooled score; the per-suite exceptions above remain the stronger evidence of interference.

Sequencing against fidelity, separated. *Sequence given* forces each step's tool to the gold plan's, keeping the policy's arguments; *arguments given* is the reverse. Episode success: best arm 0.252 free, 0.252 sequence given, 0.786 arguments given; 11M arm 0.205 free, 0.214 sequence given, 0.429 arguments given; Table 1 96M 0.000 free, 0.000 sequence given, 0.000 arguments given. Arguments buy more than sequence: argument fidelity binds harder than sequencing on the arm the abstract quotes; planning is not exonerated, merely the smaller measured constraint. Substitution is post hoc, so the sequence probe could be structurally understated (arguments were generated for the model's own tool); two checks bound that concern. First, the best arm's freely chosen tool already matches the gold plan's on 93% of steps, so the sequence substitution is close to a no-op by construction rather than by unfairness. Second, a conditioned variant states the required tool in the observation and lets the policy re-decode the whole call for it: completion is 0.252, again indistinguishable from the free run, so the attribution survives conditioning.

The recipe is not ours alone. The same episode decisions at the same 12.5% share, added to two open models' own fine-tuning and evaluated through their own chat templates: SmolLM2-135M moves 0.000→0.086 and Qwen3-0.6B 0.000→0.543. The zero-without-episodes result generalises — both controls emit near-perfect calls (schema 0.99/1.00, copy fidelity 0.97/1.00) and complete nothing, choosing the right tool on only 21–23% of steps — and the binding constraint inverts with scale: the sub-100M arms are limited by verbatim argument fidelity (0.20–0.34), the open models by tool choice (0.35 and 0.70 agreement after training, at copy $\geq$0.97). Under each family's own post-training configuration (corpus base, budget and serialisation differ, and the LocalAgent arm consumes roughly ten times the episode-gradient exposure, 1,500 steps at batch 32 against 600 at batch 8, so this prices recipes, not architectures) the 10.5M LocalAgent arm attains 0.205 against the 135M SmolLM2 arm's 0.086. Each open arm is one training seed; Qwen3-0.6B's 0.543 carries a task-level Wilson 95% interval of [0.475, 0.609]. The single-call ledger after episode SFT, on function-name match against each model's own Table 1 control: SmolLM2-135M moves +2.4/−2.0 points on the author/projection scope means and Qwen3-0.6B +0.3/−5.7, the losses concentrated in ToolSandbox (−23.5) and AgentNet (−14.7); at 0.6B the 12.5% share is not free. On the fully held-out three-goal composition both trained open arms also complete nothing (0.000 and 0.000); Qwen3-0.6B reaches 54% of its milestones there without ever assembling the full sequence, so the compositional ceiling is not an artefact of our family.

The agentic arm. Adding the episode decisions to the same mixture, midtrain parent and 1,500-step budget moves the 11M rung from 0.0% to 20.5% episode success. The state machine is learned: schema-satisfying calls rise from 0.5% to 82.1% of steps, state-moving calls from 0.1% to 59.1%. Exact reproduction of shown text is not, at 29.6% over 1,861 text-bearing steps; the families rank by how much each needs: recovery 100%, browser 10%, notion 30%, email 3%. The dominant error, `send_args_do_not_match_draft` (157), is text disagreeing with the policy's draft. Argument fidelity is the largest measured error source; the oracle decomposition separates it from sequencing.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state only measured claims; each maps to a section (§3–§6) and every suite carries an explicit claim boundary (Appendix A).

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 states the claim boundaries; Appendix A adds per-suite scope notes, Appendix D documents harness faults, and Appendix E limits task completion to the controlled simulator and reports the held-out failure.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is empirical and contains no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Appendix A pins datasets, splits, prompts, the single evaluation harness, and the shared fine-tuning recipe; all tables are generated from run receipts.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All datasets are pinned public artifacts (Appendix A) with SHA-256 manifests in the receipts; the harness, training and evaluation code, the episode simulator with its task-suite generator, every configuration and every receipt the paper reads are attached as supplementary material.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section 3 and Appendices A–B give the evaluation configuration and the distinct matched, shipped, vendor, and episode fine-tuning recipes; Appendix B tabulates their update type, rate, batch, length, steps, and corpus.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Single-call noise floors use five full-chain runs of one 10.5M configuration (§6, Appendix A). Episode claims distinguish task-bootstrap intervals for fixed checkpoints from three-seed ranges available for only two procedures (§5, Appendix E); other episode cells are labelled single-checkpoint estimates.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix C states the two H100 NVL pods and reports measured timing where receipts contain it (the controlled 1,599-step corpus arms take 20–21 minutes each). The paper does not claim a nonexistent per-arm wall-clock table for every historical run.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The research conforms to the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

Justification: This is a foundational measurement study of sub-billion on-device agents; no deployment or capability with direct societal-impact pathways is released.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No models or data with high misuse risk are released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix A names the non-standard licenses in use (the Apple Sample Code License for ToolSandbox and CC-BY-4.0 for MCP-Atlas) and its pretraining-source table records the corpus licenses; per-asset manifests with SHA-256 pins accompany the supplementary receipts.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The new evaluation projections (ToolSandbox, MCP-Atlas, Mobile-Actions) are documented in Appendix A with their conversion rules and claim boundaries; the converted files are derivable from the pinned sources.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: No crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

Justification: No human-subject research requiring IRB approval.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: An LLM assistant was used for engineering the experiment pipeline and drafting text; all experimental design decisions are documented in the paper and every number derives from the measured run receipts.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.